

Tribhuvan University

Institute of Engineering

Pulchowk Campus

A

Report

On

De-LlMIT:Decentralized Large-language Model Inference and Tuning

Submitted By:

Aavash Chhetri (PUL077BCT004)

Kushal Paudel (PUL077BCT039)

Mukti Subedi (PUL077BCT048)

Submitted To:

Department of Electronics & Computer engineering

April, 2025

Acknowledgments

We would like to express our heartfelt gratitude to the **Department of Electronics and Computer Engineering** at Pulchowk Campus for providing us with the invaluable opportunity to undertake our major project. The unwavering support, guidance, and encouragement from the department have been instrumental in shaping the trajectory of our work.

Special appreciation goes to **Assistant Prof. Bibha Sthapit** for her inspiring words, encouragement, and guidance. Her belief in our potential has been a significant motivator, and we are truly thankful for her continuous support throughout this endeavor.

The feedback and suggestions from the department have greatly contributed to refining our approach and enhancing our perspective on the project. We are confident that with the continued support from the Department of Electronics and Computer Engineering and the guidance of Assistant Prof. Bibha Sthapit, our project will continue to grow and succeed.

Thank you to the department and Assistant Prof. Bibha Sthapit for their pivotal roles in the early stages of our project. We eagerly look forward to further collaboration and the discoveries that lie ahead.

Abstract

The traditional process of Large Language Model (LLM) inference and fine tuning includes use of large cluster of heavy usage GPUs, including the cluster of Nvidia A100s and H100s GPUs. This can be replaced using a large cluster of distributed edge computing resources which can be used to infer LLMs using low-powered and economic edge resources. The project aims to develop a decentralized inference network designed to perform AI model inference and fine tuning using consumer grade devices. De-LlMIT leverages peer-to-peer (P2P) and multi layered architecture for the inference and fine tuning of large language models. By distributing model inference tasks across participating nodes, De-LlMIT mitigates the need for centralized servers, thereby reducing operational costs while enhancing scalability. This project addresses critical challenges in decentralized AI inference, including fine-tuning, scalability, and efficient resource utilization.

Keywords: Decentralized AI, Peer-to-Peer Networks, Volunteer Computing, Model Parallelism

Contents

Acknowledgements **ii**

Abstract **iii**

Contents **vi**

List of Figures **vii**

List of Abbreviations **viii**

Copyright **x**

1 Introduction **1**

1.1 Background 1

1.2 Problem statements 1

1.3 Objectives 2

1.4 Scope 2

2 Literature Review **3**

2.1 Related Theory 3

2.1.1 Volunteer Computing 3

2.1.2 Distributed Machine Learning: Federated Learning and Model-Distributed ML 4

2.1.3 Distributed Hash Tables and Networking 4

2.1.4 Large Language Models: Memory Requirements, Quantization, and Challenges 5

2.1.5 Machine Learning in Browser Based Environment 6

2.2 Related work 6

3 Project Methodology **8**

3.1 Block Diagram of the System 8

3.1.1 Inference of Large-Language Models 8

3.2 Working Flow of the System 14

3.2.1 Distributed Hash Table (DHT) Initialization 14

3.2.2 Initialization and Joining the Network . . . 15
3.2.3 Client Request Initialization . . . 15
3.2.4 Fine-Tuning in De-LlMIT Architecture . . . 16

4 System Design **19**
4.1 Layer Descriptions . . . 19
4.1.1 Browser Instances . . . 19
4.1.2 Terminal Instances . . . 20
4.1.3 Monitor Clients . . . 20

5 Implementation Details **21**
5.1 Problem Formulation . . . 21
5.1.1 Inference Objective . . . 21
5.1.2 Fine-Tuning Objective . . . 21
5.2 Model Sharding . . . 22
5.2.1 Model Sharding in Delimit . . . 22
5.2.2 Splitting the Pretrained Weights . . . 23
5.3 DHT and Logits Sharing . . . 24
5.4 Llama Implementation . . . 24
5.5 Sharding Llama Model . . . 27
5.5.1 Weights loading . . . 27
5.6 Fine Tuning . . . 27
5.6.1 LoRA . . . 27
5.6.2 Setting up model for LoRA fine tuning . . . 28
5.6.3 Preparing DataSets and Dataloaders . . . 28
5.6.4 Distributed FineTuning . . . 29
5.7 Multi-Layer WebSocket Communication System . . . 30
5.7.1 Server-Side Implementation . . . 31
5.7.2 Client-Side Implementation . . . 32
5.8 Experimental Setup for PEFT . . . 33

6 Results **34**
6.1 Experimentation . . . 34
6.1.1 Experiment with Different Bandwidths . . . 34
6.1.2 Fine-Tuning . . . 35
6.1.3 Discussion . . . 37

7 Conclusion **40**

REFERENCES **43**

List of Figures

3.1 Overview of inference in De-LlMIT. 8
3.2 Workflow of the De-LlMIT system. 14

4.1 System Design 19

5.1 Architecture of a generic LLM 22
5.2 Vizualization of different layers in a Model 23
5.3 Llama3.2-1B Architecture 25
5.4 SwiGLU FFN 26
5.5 Multi-Head Grouped Query Attention 26
5.6 LoRA 28
5.7 Alpaca Style Prompt Formatting 29
5.8 A single step in fine-tuning 30

6.1 Average inference time per 50 token 35
6.2 Average training time per step 35
6.3 Validation Loss per step 36
6.4 Training Loss per step 36

List of Abbreviations

De-LlMIT	Decentralized Large Language Model Inference and Tuning
LLM	Large Language Model
P2P	Peer-to-Peer
GPU	Graphics Processing Unit
DHT	Distributed Hash Table
DNN	Deep Neural Network
NAT	Network Address Translation
STUN	Session Traversal Utilities for NAT
TURN	Traversal Using Relays around NAT
BIN	Browser Instance Node
TIN	Terminal Instance Node
MCN	Monitor Client Node
ONNX	Open Neural Network Exchange
RAM	Random Access Memory
vRAM	Video Random Access Memory
FFN	Feed-Forward Network
GPT	Generative Pre-trained Transformer
LLAMA	Large Language Model Meta AI
HPC	High-Performance Computing
HTTP	Hypertext Transfer Protocol
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
AI	Artificial Intelligence
API	Application Programming Interface
ML	Machine Learning
BLOOM	BigScience Large Open-science Open-access Multilingual Language Model
H100	NVIDIA Hopper-100 GPU
A100	NVIDIA Ampere-100 GPU
SSD	Solid State Drive

fp16	16-bit Floating Point Precision
SiLU	Sigmoid Linear Unit
RoPE	Rotary Positional Embedding
PEFT	Parameter-Efficient Fine-Tuning
AdamW	Adaptive Moment Estimation with Weight Decay
AWS S3	Amazon Web Services Simple Storage Service
IPv6	Internet Protocol version 6
MbPS	Megabits Per Second
json	JavaScript Object Notation
LoRA	Low-Rank Adaptation

Copyright

The author has agreed that the Library, Department of Electronics and Computer Engineering, Pulchowk Campus, and Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering for any use of the material of this project report. Copying publication or the other use of this report for financial gain without the approval of to the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, and the author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head
Department of Electronics and Computer Engineering
Pulchowk Campus, Institute of Engineering, TU
Lalitpur, Nepal.

1. Introduction

The rapid advancements in artificial intelligence and large language models (LLMs) have revolutionized natural language processing, data analytics, and human-computer interaction. However, centralized LLM inference methods face challenges such as scalability, latency, privacy concerns, and high hardware requirements, making them inaccessible to many users and individual researchers. To address these issues, we introduce "De-Llmit: Decentralized Large Language Model Inference and Tuning," which leverages decentralized computing to perform LLM inference directly within consumer devices. By distributing model computations across a network of nodes, De-Llmit enhances scalability, and protects user privacy without the need for centralized infrastructure or complex installations. This approach democratizes access to LLMs, enabling efficient and private model inference on consumer-grade hardware, thus redefining the accessibility and usability of AI technologies.

1.1 Background

Large language models (LLMs) like GPT-4 and LLaMA are increasingly complex, with models such as GPT-4 having 175 billion parameters and requiring multiple gigabytes of storage even with reduced precision (16-bit). Managing such models demands high-end hardware and incurs significant costs[1], [2]. Personal and home computers are typically inadequate for these models due to their substantial memory and computational requirements. A 175B model requires about 350GB of memory in fp16 settings just to load the whole model. A typical consumer has neither VRAM nor RAM required to store the model. According to researchers at [3] BLOOM-176B took 5.5 seconds per token using fast RAM offloading and 22 seconds per token using SSD offloading which is unusable for many purposes.

Decentralized AI approaches offer a solution by leveraging distributed computing power from end-user devices. Studies, such as Lian et al. [4], show that this method can alleviate centralized server burdens, reduce costs, and make AI technology more accessible.

1.2 Problem statements

Some of the problems with the traditional approach:

1. **Centralized Servers**: Traditional AI inference relies heavily on centralized servers, leading to single points of failure, high latency, and dependence on server availability and performance.

2. **Memory Constraints**: LLMs like LLaMA-2 or GPT variants require hundreds of gigabytes of memory to store model parameters. Consumer-grade GPUs and CPUs often lack sufficient memory to load the entire model.

$$Required\,Condition : RAM + vRAM \geq Model\,Memory\,Requirement$$

3. **Computation Time**: Even if the model fits into memory, performing inference on large models requires significant computational power, leading to longer inference times and higher latency.

1.3 Objectives

Our objective is to try to overcome the drawbacks of the traditional system of model training by building a peer-to-peer network solution comprising of the following main features:

1. An Model Parallel System: Develop a system that enables LLM inference and finetuning in an decentralized environment by sharding the LLM model and loading partial models in different devices to carry out inference and tuning by using an exhaustive set of clients that form a whole model.

2. A Decentralized Model inference and tuning: Develop a system that enables AI language model inference using a cluster of low compute end devices, eliminating the need for centralized servers. Utilize the computational power of client devices for efficient and seamless inference processes and tuning.

1.4 Scope

1. Distributed AI Inference: Develop AI models that run directly within terminal instances or web browsers.

2. Client Finetuning: Develop a system to enable finetuning by using decentralized computing powers of the client terminals in the decentralized network.

3. Decentralized Network: Create a network of computers consisting of three layered architecture that can handle inference, fine tuning in the client terminal layer, fault tolerance and communication between terminals to exchange metadata and logits.

4. Ease of Use and Accessibility: Enable easy participation for users with different technical backgrounds. Promote widespread adoption through user-friendly interfaces and minimal setup requirements.

2. Literature Review

2.1 Related Theory

2.1.1 Volunteer Computing

1. The paper *Hivemind: Decentralized Deep Learning for Large-Scale Models* [5], [6] introduces a decentralized framework for training and serving large-scale models across a distributed network of devices. It addresses the limitations of centralized AI training by enabling multiple participants to collaboratively train models without sharing their raw data. Hivemind leverages peer-to-peer communication, consensus algorithms, and advanced synchronization techniques to coordinate training across nodes, making it possible to scale deep learning tasks beyond the capacity of single organizations or data centers

2. *Petals: Collaborative Inference of Large Models like BLOOM* [3] presents a framework for distributed inference of large language models through a collaborative peer-to-peer network. It allows users to run inference on models, such as BLOOM, by connecting to a network where each participant hosts parts of the model. This decentralized approach reduces the computational burden on individual users by sharing the workload across multiple nodes, making it feasible to run large models even on limited hardware. Petals aims to make advanced model inference accessible, scalable, and more cost-effective by leveraging community-driven participation.

3. *Folding@Home* [7] describes a distributed computing project that utilizes the idle computational power of millions of volunteer devices worldwide to simulate protein folding and study molecular dynamics. By distributing small computational tasks to participants' machines, the project tackles complex problems in biology and medicine, such as understanding protein misfolding in diseases like Alzheimer's and developing potential treatments. This collaborative model accelerates research by aggregating vast amounts of processing power, demonstrating the potential of decentralized computing to solve large-scale scientific challenges.

2.1.2 Distributed Machine Learning: Federated Learning and Model-Distributed ML

1. The article *The Future of Consumer Edge-AI Computing* by Stefanos Laskaridis et al. [8] explores the trajectory and potential of edge AI computing, particularly focusing on consumer applications. The paper discusses the challenges and opportunities associated with deploying AI models on edge devices, emphasizing the need for efficient resource management and model optimization. The authors provide a comprehensive overview of current technologies and future directions, highlighting the impact of edge AI on various consumer sectors.

2. The paper *Federated Learning: Challenges, Methods, and Future Directions* [9] provides a comprehensive overview of federated learning, a decentralized approach where multiple clients collaboratively train a global model without sharing their raw data. It addresses key challenges such as data heterogeneity, privacy preservation, communication efficiency, and security concerns. The paper reviews various methods to tackle these issues, including advanced aggregation techniques, differential privacy, and secure multi-party computation

3. *Model-Distributed Machine Learning* [10]explores a distributed approach to training and deploying machine learning models by partitioning models across multiple devices or nodes. Unlike traditional data-parallel methods, this approach distributes parts of the model itself, allowing each node to handle only a portion of the overall computation. This model distribution can optimize resource usage, reduce memory requirements, and enable training and inference on hardware with limited capacity. The paper discusses the architecture, challenges, and potential benefits of this approach, emphasizing its ability to scale large models efficiently and enhance collaboration in multi-device environments.

2.1.3 Distributed Hash Tables and Networking

1. *Kademlia: A Peer-to-Peer Information System Based on the XOR Metric* [11] introduces the Kademlia DHT, a fundamental framework for peer-to-peer (P2P) networks. The authors present an efficient XOR-based routing mechanism for locating data across a decentralized network. This routing approach is crucial for enabling the DHT in systems like *De-Llmit*, where peers need to exchange intermediate model outputs efficiently.

2. *P2P Networking and DHTs in Decentralized Systems* [12] examines the use of peer-to-peer (P2P) networks and Distributed Hash Tables (DHTs) as foundational technologies for decentralized systems. It explores how P2P networking enables direct communication between nodes without relying on centralized servers, enhancing scalability and fault tolerance. DHTs are highlighted as crucial for efficient data lookup and resource management in these networks, providing a structured way to distribute and retrieve information across nodes.

3. **WebSocket in Decentralized and Volunteer Computing:** Decentralized frameworks rely on efficient communication protocols to coordinate distributed tasks. For instance, Hivemind[5] leverages peer-to-peer communication for collaborative model training across devices. While the paper focuses on consensus algorithms, WebSocket's persistent connections and low overhead make it ideal for synchronizing model updates and gradients in real time, minimizing delays caused by repeated HTTP handshakes. Similarly, Folding@Home[7] distributes computational tasks across volunteer devices, where WebSocket's ability to maintain open channels ensures rapid task allocation and result aggregation, critical for scaling complex simulations like protein folding.

2.1.4 Large Language Models: Memory Requirements, Quantization, and Challenges

1. *Scaling Up Language Models: GPT-3* [13] introduces GPT-3, one of the largest and most powerful language models ever created, with 175 billion parameters. It demonstrates how scaling up model size significantly enhances the performance of language models across a wide range of tasks without task-specific training. The paper explores GPT-3's capabilities in natural language understanding and generation, showcasing its ability to perform complex tasks such as translation, question answering, and code generation with minimal prompts.

2. *Quantization and Efficient Inference in Large Models* [14], [15] discusses techniques for reducing the computational and memory demands of large language models through quantization. Quantization involves converting model weights and activations from high-precision formats to lower precision, such as 8-bit integers, without significantly compromising accuracy. This process enables more efficient inference, making it feasible to deploy large models on resource-constrained devices like mobile phones and edge servers.

3. *Memory-Efficient Training of Transformers* [16]presents strategies to optimize the memory usage of transformer models during training. It focuses on techniques such as

mixed precision training and gradient checkpointing, which reduce the memory footprint while maintaining performance. By enabling the training of larger models or increasing batch sizes without requiring extensive hardware resources, these methods make it feasible to train state-of-the-art transformers efficiently.

2.1.5 Machine Learning in Browser Based Environment

1. The paper *WebInf: Accelerating WebGPU-based In-browser DNN Inference via Adaptive Model Partitioning* [17] proposes a novel approach to optimize deep neural network (DNN) inference within web browsers using WebGPU. It introduces adaptive model partitioning, which dynamically splits models between the client device and external servers based on available resources and network conditions. WebInf aims to enable efficient and scalable DNN inference directly in the browser, making advanced AI applications more accessible without relying on powerful local hardware.

2. *WebDNN: Fastest DNN Execution Framework on Web Browser* [18] by Mitsuhisa Sato et al. introduces WebDNN, a framework designed to execute deep neural network (DNN) models efficiently within web browsers. Leveraging technologies like WebAssembly and WebGPU, WebDNN achieves remarkable performance improvements, making it the fastest known framework for in-browser DNN execution at the time. This advancement opens up new possibilities for deploying machine learning models directly to users without the need for server-side computation.

2.2 Related work

1. On-Device Machine Learning with TensorFlow.js Smilkov et al. (2019) explored the capabilities of TensorFlow.js for running machine learning models within web browsers. Their work demonstrated that complex neural networks could be executed efficiently on client-side devices, paving the way for decentralized inference applications. The study highlighted the potential for interactive and real-time AI applications without relying on centralized servers [19].

2. Blockchain-Based Incentive Mechanisms for Decentralized Computing Wu et al. (2018) proposed a blockchain-based framework to incentivize users to share their computational resources. This approach ensures transparency and security in tracking contributions and distributing rewards. The study provided insights into designing effective incentive schemes, which are essential for the success of decentralized inference systems [20].

3. Federated Learning: Challenges, Methods, and Future Directions Wen et al. (2023) reviewed the advancements in federated learning, discussing its potential to preserve data privacy and leverage distributed computational resources. Although focused on training, the principles of federated learning are relevant to decentralized inference, emphasizing the benefits of local data processing and decentralized architectures [21].

4. Efficient Decentralized Computing for AI Applications Lei et al. (2020) investigated various decentralized computing frameworks for AI applications, analyzing their performance and scalability. The study highlighted the advantages of decentralized systems in handling large-scale AI workloads and provided a comprehensive evaluation of different approaches, including browser-based inference [22].

5. Distributed Inference and Fine-Tuning for Large Language Models
   A distributed inference framework for large models like GPT and DALL-E was proposed by Borzunov et al. (2023). This framework introduces a method for decentralized inference, allowing models to be split across devices, improving efficiency. This approach minimizes network latency and introduces fault-tolerant algorithms, ensuring reliable generation even with device or network failures [23].

3. Project Methodology

3.1 Block Diagram of the System

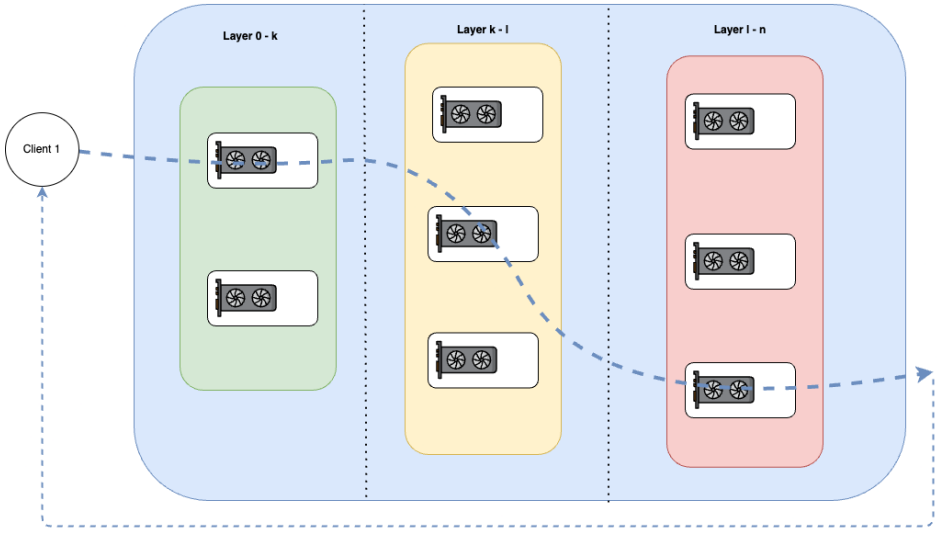


Figure 3.1: Overview of inference in De-LlMIT.
Client-1 wants to run inference task but can't afford (hardware limitations). It creates a inference request to the system. A sequence is generated by the system which specifies the path of inference and the path is taken.

De-LlMIT is designed to perform two main tasks: inference of prompt and fine-tuning of large models which can't be loaded in a single consumer grade device. In this section, we outline the design of De-LlMIT, both from inference point of view and fine tuning point of view.

3.1.1 Inference of Large-Language Models

Large-Language Models (LLMs) are typically made up of multiple layers of blocks such as Transformer blocks, Attention-Head blocks, and Feed-Forward Networks (FFNs). The large number of such blocks, interconnected in a structured manner, makes LLMs a powerful tool for natural language understanding, generation, and various other downstream tasks. However, this sheer scale poses a significant challenge when deploying these models on consumer-grade hardware due to the immense memory and computational requirements.

Model Sharding in De-LlMIT

The primary innovation in De-LlMIT is model sharding, which divides the LLM into smaller, manageable parts that can be independently processed across a distributed network of devices. Each shard contains a subset of the model's layers—such as a specific number of Transformer or Attention blocks which can be loaded into the memory of a standard GPU or CPU. Following algorithm describes the model sharding process:

Algorithm 1 Sharding Model Weights for Distributed Loading

Require: Pretrained model prefix P, number of layers L, sharding range $[s, e]$

Ensure: Sharded model with subset of layers

function ShardWeights(P, s, e)
    $M \leftarrow$ Empty Model Dictionary
    if $s = 0$ **then**
        M[embed_tokens] $\leftarrow$ InitializeEmbedding(P)
    end if
    for $i \leftarrow s$ to $e - 1$ **do**
        $W_i \leftarrow$ LoadLayerWeights(P, i)
        $M[\text{layer}_i] \leftarrow$ InitializeTransformerLayer(W_i)
    end for
    if $e = L$ **then**
        M[norm] $\leftarrow$ InitializeLayerNorm(P)
        M[lm_head] $\leftarrow$ InitializeLMHead(P)
    end if
    return M
end function

Distributing Model Weights

After shrading the model, each shard weight was stored as a separate repository on the HuggingFace Model Hub. The configuration file and the model weights are uploaded independently for each shard. This approach allows for efficient distributed training and inference by only loading the necessary parts of the model when needed. The dynamic loading of model is done with the following algorithm:

Algorithm 2 Load Model from HuggingFace Model Hub

Require: `pretrained_model_prefix`, `start_layer`, `end_layer`

for $i \leftarrow$ `start_layer` to `end_layer` $- 1$ **do**
  Set `repo_name` $\leftarrow$ `pretrained_model_prefix` concatenated with "-" and i
  if $i ==$ `start_layer` **then**
    Download `config.json` from `repo_name`
    Parse `config.json` to obtain `config_dict`
  end if
  Download `pytorch_model.bin` from `repo_name`
  Load the weights into `state_dict` from `pytorch_model.bin`
  if $i ==$ `start_layer` **then**
    Initialize the model with `config_dict` and the specified layer range
  end if
  Update the model with `state_dict` for the current shard
end for
return the fully loaded model

The algorithm remaps a global state dictionary into a shard-specific dictionary for a large model by selectively including only the parameters that belong to the shard's designated layer range. It begins by initializing an empty dictionary for the shard. As it iterates over each key-value pair in the global state dictionary, it checks if the key corresponds to an embedding layer, a transformer layer, or a final module. The embedding layer is included only if the shard is the first one, while transformer layers are added only if their indices fall within the shard's start and end boundaries. Additionally, final modules such as normalization layers or the language modeling head are included only if the shard is the last segment of the model. This method ensures that each shard contains only the necessary parameters, facilitating efficient distributed training and inference by partitioning the overall model into manageable pieces.

Algorithm 3 Remap Global State Dictionary for Model Shard (Full Model)

Require: Global state dictionary θ_{global}, shard start layer s, shard end layer e, total number of layers L

Ensure: Remapped state dictionary θ_{shard}

Initialize $\theta_{\text{shard}} \leftarrow \{\}$
for each (k, v) in θ_{global} **do**
    if key k corresponds to the embedding layer **and** $s = 0$ **then**
        $\theta_{\text{shard}}[k] \leftarrow v$ ▷ Keep embed_tokens if this is the first shard.
    else if key k corresponds to a transformer layer **then**
        $i \leftarrow \text{ExtractLayerIndex}(k)$
        if $s \leq i < e$ **then**
            $\theta_{\text{shard}}[k] \leftarrow v$ ▷ Include layers in the current shard range.
        end if
    else if key k corresponds to final modules (e.g., norm or lm_head) **and** $e = L$ **then**
        $\theta_{\text{shard}}[k] \leftarrow v$ ▷ Keep final normalization and language modeling head if last shard.
    else
        continue ▷ Skip keys not applicable to this shard.
    end if
end for
return θ_{shard}

Forward Pass with the Distributed Model

Each consumer-grade device is responsible for processing the forward pass through its assigned shard. After completing its computations, the device forwards the resulting intermediate tensor (i.e., the logits) to the next device in the sequence. In this way, the inference process moves from one shard to the next, mimicking the sequential layer computation typical of an LLM running on a single, larger device.

To ensure proper sequential execution, De-LlMIT orchestrates the flow of data across devices. When an input prompt is received, it is tokenized and fed into the first shard. The logits produced by this shard are transferred to the next shard in the sequence, where further computation occurs. This process repeats until the entire model has been traversed, and the final logits are generated as output. Following is the algorithm that describes the overall working:

Algorithm 4 Distributed Forward Pass with Byte-Based Tensor Transmission

Require: Received byte tensor B_{in} from machine M_{src}, model M with layer range $[s, e]$

Ensure: Processed byte tensor B_{out} sent to machine M_{dst}

function FORWARDPASS(B_{in}, M, s, e)
  $T \leftarrow$ ConvertBytesToTensor(B_{in}) ▷ Convert received bytes to tensor
  if $s = 0$ **then**
    $T \leftarrow$ EmbedTokens(M, T) ▷ Apply embedding layer if first shard
  end if
  for $i \leftarrow s$ to $e - 1$ **do**
    $T \leftarrow$ ApplyLayer($M[\text{layer}_i], T$) ▷ Forward pass through each transformer layer
  end for
  if $e =$ TotalLayers(M) **then**
    $T \leftarrow$ ApplyLayerNorm(M, T)
    $T \leftarrow$ ApplyLMHead(M, T) ▷ Apply final layer normalization and LM head
  end if
  $B_{out} \leftarrow$ ConvertTensorToBytes(T) ▷ Convert output tensor to bytes
  Send B_{out} to machine M_{dst} ▷ Transmit processed tensor to next machine
  return B_{out}
end function

The bottleneck here can be a central server trying to manage the layer shrads and the helping the logits to transfer from one client to another. De-LlMIT tackles this by employing a partially distributed network type of architecture. 3 different types of nodes are employed in the De-LlMIT viz. Monitor Client Nodes, Terminal Instance Nodes and Browser Instance Nodes.

Monitor Client Nodes These are the nodes which is primarily responsible only for maintaining the network. They are the nodes which will remain active at all times, and can be reached by any other nodes to join the network. These nodes maintains a Distributed Hash Table (DHT) which contains the list of other monitor client nodes, and the active peers, layers the peer holds. It also holds a Hash Table which holds the address of the peers connected to itself. The heavylifting of the transfer such as for the transfer of logits are primarily done using the WebSocket which is maintained across the client and the MCNs. DHT is responsible for managing the layers and client addresses with in different MCNs.

Terminal Instance Nodes These nodes are the terminal instance of the Medium-High Performance Computing Devices which can contribute to the inference and fine-tuning i.e.

loads the shred of model layers and performs the forward pass. They are client type of nodes responsible for computation and aren't accountable for maintaining the overall system processes.

Browser Instance Nodes These are special types of nodes, which is first used by DeLlMIT. This type of nodes are present in the browsers of consumer-grade devices. It utilizes the different openly available services like kaggle and google colab to run inference/tuning tasks using the web browser environment.

The *Monitor Client Nodes* are responsible for maintaining the DHT in addition to creating a serialized path of inference with in the network. This serialized path should ensure a proper inference of the model ensuring that the set of models in the sequence form a mutually exhaustive and exhaustive set of the original model. The architecture incorporates fault-tolerance by employing multiple nodes to load the same layer, having consistent backbone MCN to sustain the network.

The majority of the connections within the system is handled using the WebSocket Protocol (RFC6455)[24]. This is done, since this set up required full duplex communication with minimal latency and needs to transfer large bandwidth of data i.e. logits.

Multi-Layer WebSocket Communication System

The system implements a multi-layer WebSocket communication framework that leverages asynchronous programming for real-time, dynamic message processing and performance tracking. On the server side, dedicated components manage client connections and worker assignments—each worker being responsible for a specific processing layer—with a central loop that iteratively refines messages by dispatching them to available workers. Clients establish connections through a binary handshake that encodes both their processing layer and unique identifier, then enter a processing loop to handle incoming messages, perform necessary data conversions, and log detailed timing metrics for performance analysis. This design ensures efficient load balancing, robust error handling, and smooth communication in line with WebSocket protocols as defined in RFC6455. Further is explained in Implementation Details.

3.2 Working Flow of the System

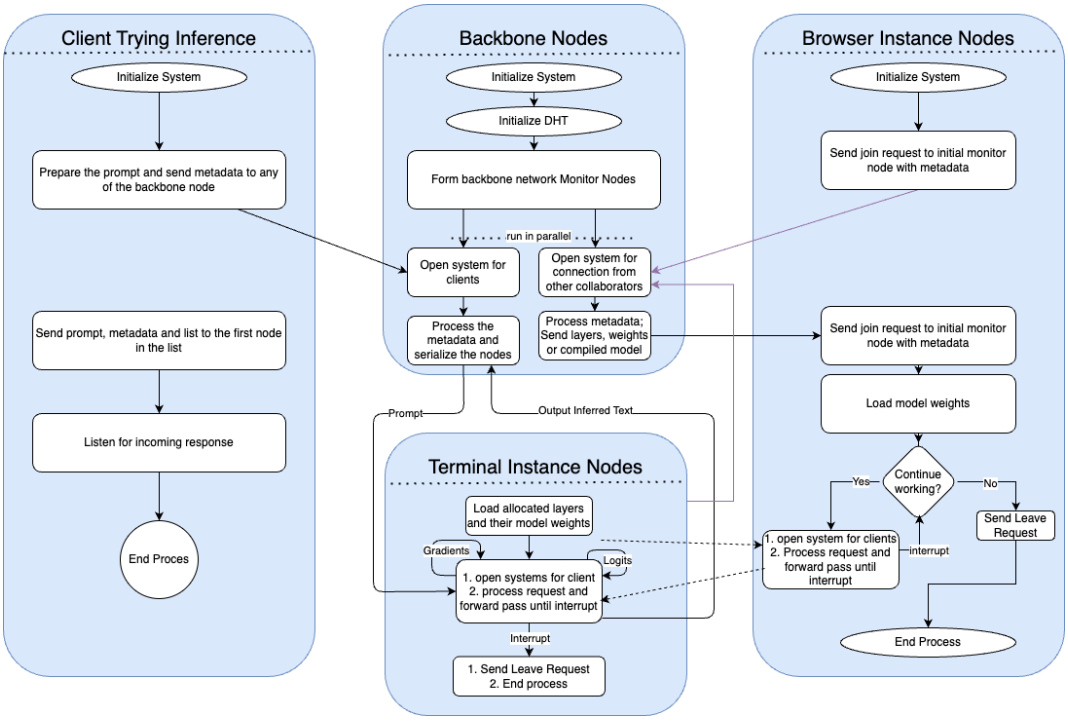


Figure 3.2: Workflow of the De-LlMIT system.

Figure showing interaction between Client Nodes, Backbone Nodes, Browser Instance Nodes, and Terminal Instance Nodes. Backbone Nodes are initialized first and distribute layer weights to newly joining Browser or Terminal Instance Nodes upon request. Once integrated, these nodes become available for client use. Inference begins when a client sends a request, which is routed by Backbone Nodes that serialize the inference path. Distributed inference is performed across consumer devices using sharded model segments, with Browser and Terminal Instances executing forward passes.

3.2.1 Distributed Hash Table (DHT) Initialization

The DHT is employed to facilitate peer discovery. Each peer announces its presence and registers on the DHT, making its computational resources available for decentralized inference. The DHT stores the address of the BINs and TINs along with the layers they hold. The data exhange is done in real time using the WebSocket Protocol. This decentralized storage mechanism allows peers to communicate without relying on a central server, ensuring scalability and fault tolerance.

They are primarily initialized in the monitor client nodes. The Monitor Client Nodes are responsible for maintaining the DHT, and any new node, any of the 3, which wants to the

join the network should join the network via one of the stable monitor client node.

3.2.2 Initialization and Joining the Network

The initialization of De-LlMIT involves the configuration of nodes and their integration into the network. Two key processes take place during this phase: *load shraded models* and *peer distribution*.

Each peer that joins the network announces its availability by registering with the Monitor Client Nodes, providing details about the shards they hold. This registration process ensures that the system can route inference requests to the appropriate peers based on their capabilities.

3.2.3 Client Request Initialization

The inference process begins when a client, e.g., *Client-1*, initiates a request. This request consists of a prompt that the client wishes to process using the distributed model.

Sending a Request for Path

- *Client-1* sends an inference request, containing the prompt and other necessary metadata, to one of the **Monitor Client Nodes**.
- The MCN handles the request by identifying the path through the network required for performing the inference based on the distributed layers.

Serialized Path for Inference

- The MCN refers to the Hash Table to identify the active peers that hold the necessary model shards.
- A *serialized path* is then generated, specifying the sequence of nodes through which the inference request will pass.

This path ensures that the layers are processed in the correct order, allowing distributed execution across multiple nodes while maintaining the integrity of the model's computation.

Execution of Forward Pass and Inter-Node Communication

- *MCN* sends the input to the first node in the path, which is responsible for processing the first shard of the model.

- The first node processes the input through its assigned layers and computes the intermediate logits. It then forwards the intermediate result to the MCN and to next node in the sequence.
- This communication continues between nodes along the serialized path, with each node processing its shard and forwarding the logits until the entire model has been traversed.
- Finally, the output Inferred Text is passed pack to the client.

The communication between nodes is managed by the Monitor Client Node and is done using the WebSocket Protocol. DHT is responsible for managing the client and the layers they hold in the MCNs.

Returning the Inference Result

- The last node in the serialized path completes the computation and generates the final output.
- The final logits are returned to *MCN*, completing the inference request.
- Finally, the output is passed to the Original Client.

The distributed architecture of De-LlMIT enables the inference of large language models on consumer-grade hardware by dividing the workload across multiple peers. The final output is generated after passing through all the necessary layers of the model, distributed across the nodes in the network.

3.2.4 Fine-Tuning in De-LlMIT Architecture

Along with inference, the De-LlMIT architecture is also designed to support the fine-tuning of large models that cannot be fully loaded on consumer-grade hardware. The fine-tuning process is efficiently handled by distributing the model layers across multiple devices, while the final fully connected layer is fine-tuned on a client device. This section details the procedure for fine-tuning using this distributed system.

Initialization and Network Joining

To begin the fine-tuning process, the client node first initializes the system and joins the network. Similar to the inference process, the model is sharded into smaller, manageable layers that are distributed across the nodes within the network. Each shard is loaded onto a peer in the network in the Terminal Instance Node, depending on the computational capacity of the device.

Using the Network for Forward Pass

Once the network is initialized and the model shards are loaded across the peers, the client begins the forward pass through the network. The forward pass is similar to the inference process, where the input data is tokenized, passed to the first node in the sequence, and moves sequentially through the network. Each node computes its respective shard and passes the intermediate results (logits) to the next node. This continues until the forward pass reaches the final layer.

Fine-Tuning

After the forward pass through the distributed layers is complete, the client begins fine-tuning the fully connected layer. This is done by performing backpropagation on the final layer using the output logits received from the earlier layers. The TINs updates the weights of the layers based on the gradients calculated during backpropagation.

Backward Pass and Weight Updates

During the backward pass, the gradients are calculated for the earlier nodes using the gradients of the input to the next node, and the necessary weight updates are performed. These updates are sent back through the network, with each node updating its corresponding shard.

Algorithm:

Algorithm 5 StepTune: Single Fine-Tuning Step with Logit Sharing

Require: Input tensor `logits`; Target batch `target_batch`; Model `model`

Ensure: Gradient tensor for `logits`

Move `model`, `logits`, and `target_batch` to the designated device.
if `model.start_layer` $\neq 0$ **then**
    Set `logits.requires_grad` $\leftarrow$ True
    Retain gradients for `logits`.
end if
`output` $\leftarrow$ `model(logits)`
Initialize `optimizer` $\leftarrow$ AdamW(`model.parameters()`, lr $= 5 \times 10^{-5}$, weight_decay $=$ 0.1).
Call `optimizer.zero_grad()`.
if `model.end_layer` $=$ `model.num_hidden_layers` **then** ▷ Last layer case
    Compute loss:

$$\texttt{loss} \leftarrow \text{cross_entropy}\big(\texttt{output.flatten(0,1)}, \texttt{target_batch.flatten()}\big)$$

    Backpropagate: `loss.backward()`.
    Update parameters: `optimizer.step()`.
    return `logits.grad.data`.
else ▷ Intermediate layer case
    `output_grad` $\leftarrow$ **StepTune**($i + 1$, `output.data`, `target_batch`).
    Backpropagate using upper layer gradients: `output.backward(gradient = output_grad)`.
    Update parameters: `optimizer.step()`.
    if `model.start_layer` $= 0$ **then**
        return null.
    else
        return `logits.grad.data`.
    end if
end if

4. System Design

The architecture for the Delimit consists for 3 main layers:

1. **Browser Instances**
2. **Terminal Instances**
3. **Monitor Client**

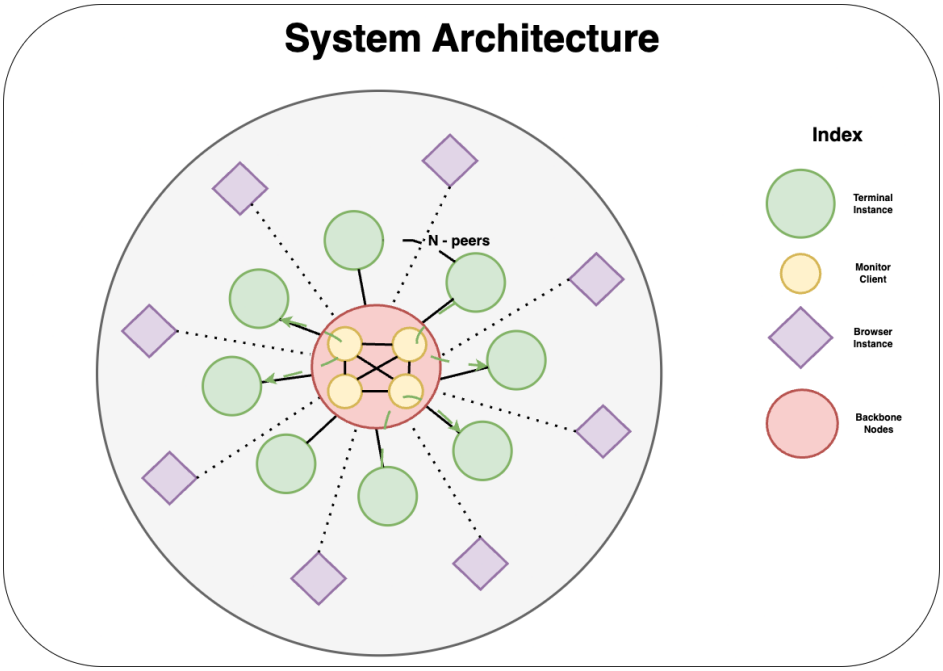


Figure 4.1: System Design

4.1 Layer Descriptions

4.1.1 Browser Instances

The outermost layer, denoted by the **Browser Instances**, consists of users interacting with the system through their web browsers. Each browser instance represents a client that communicates with inner layers or other browser instances to initialize or take part in inference. These browser instances can be seen as lightweight user endpoints that connect to the Delimit system via HTTP or WebSocket protocols. Their primary function is to provide an interface for end users to be able to use AI features without having to go through installation processes.

4.1.2 Terminal Instances

The **Terminal Instances** forms the middle layer of the system. Each terminal instance is responsible for executing specific computations related to model sharding and distributed tasks. These terminal instances are connected to each other in a peer-to-peer fashion if possible or by using the Monitor Clients as relay servers, creating a distributed network where each instance handles part of the workload. In the context of model sharding, each terminal instance might handle different layers of the GPT or llama model, with inputs being pipelined across instances.

The terminal instances can perform minor sequencing related task and also take part in finetuning of the model other than inference. They may be high performance servers in the universities or simple client terminal running on an PC.

4.1.3 Monitor Clients

At the center of the system are the **Monitor Clients**, which are responsible for orchestrating the entire process. These clients monitor the health and status of the terminal instances, ensuring that computations are progressing as expected and that instances are correctly synchronized. Additionally, monitor clients may also handle load balancing, distributing tasks to terminal instances to optimize performance.

Thse are the servers with their own IP addresses and are connected to each other in a purely P2P fashion. They also act as STUN server for NAT traversal between the terminals and/or browser instances behind NAT.

5. Implementation Details

5.1 Problem Formulation

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{K}$ denote a labeled dataset of K input-output pairs, where x_i represents the input (e.g., a prompt, sentence, or query), and y_i is the corresponding target output (e.g., label, answer, or continuation). We aim to fine-tune a pre-trained language model $\mathcal{M}_\theta$, parameterized by θ, to adapt it to a downstream task by minimizing a suitable task-specific loss function.

We consider a decentralized inference setting where the language model $\mathcal{M}_\theta$ is partitioned and distributed across n computational nodes, denoted by $\{N_1, N_2, \ldots, N_n\}$. Each node N_j holds a subset of model parameters $\theta_j \subset \theta$, such that:

$$\mathcal{M}_\theta = \bigcup_{j=1}^{n} \mathcal{M}_{\theta_j} \tag{5.1}$$

The overall model $\mathcal{M}_\theta$ is executed collaboratively across the n nodes to perform inference on an input x. This setting is suitable for scenarios where resources are distributed (e.g., edge devices, peer-to-peer systems, federated clusters) and centralized computation is infeasible or suboptimal.

5.1.1 Inference Objective

Given an input x, the goal is to produce an output $\hat{y}$ by orchestrating partial computations across nodes. The prediction process can be formulated as:

$$\hat{y} = \text{Decode}\left(\mathcal{M}_\theta(x)\right) = \text{Decode}\left(\mathcal{M}_{\theta_n}\left(\cdots \mathcal{M}_{\theta_1}(x) \cdots\right)\right) \tag{5.2}$$

or, can also be observed from the probabilistic view point as calculating the posterior probability given x, which is:

$$\hat{y} = \arg\max_y \mathbb{P}_\theta(y \mid x) = \arg\max_y \mathbb{P}_{\{\theta_j\}_{j=1}^{n}}(y \mid x) \tag{5.3}$$

5.1.2 Fine-Tuning Objective

The fine-tuning process involves optimizing model parameters θ to minimize the empirical risk:

$$\mathcal{L}_{\text{fine-tune}}(\theta) = \frac{1}{K} \sum_{i=1}^{K} \mathcal{L}_{\text{task}}\left(\mathcal{M}_\theta(x_i), y_i\right) \tag{5.4}$$

5.2 Model Sharding

Model Sharding is essential for this process, where a language model $\mathcal{M}_\theta$ is divided into different contiguous layers $\mathcal{M}_{\theta_j}$. When inference or training is performed, a sequence of sharded models is taken, forming an ordered and exhaustive set of the entire model.

$$\mathcal{M}_\theta = \mathcal{M}_{\theta_n} \circ \mathcal{M}_{\theta_{n-1}} \circ \cdots \circ \mathcal{M}_{\theta_1} \tag{5.5}$$

5.2.1 Model Sharding in Delimit

Model Sharding in De-llmit starts with the sharding of LLMs. It uses the base class from the `Hugging Face Transformers` library.

An LLM consists of an embedding layer at the beginning, which converts tokens into embedding vectors. It is followed by several transformer layers, and at the end, there is a normalization layer and a head layer. Each of these layers can be sharded and work independently to produce output once input from the preceding layer is provided.

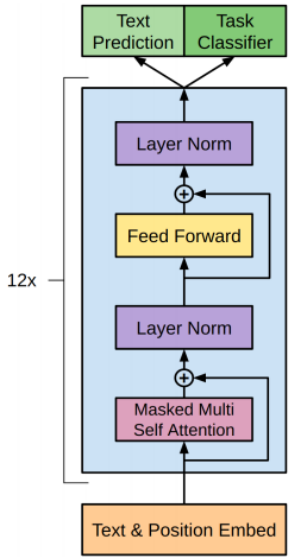


Figure 5.1: Architecture of a generic LLM
[25]

To shard the model, we create a `CustomLLMModel` using the base class. This class uses an initialization function that takes the start layer, end layer, and the total number of layers in the desired model as input. This initializes the custom model with a specific shard of layers from the start to the end layer. If the start layer is 0, it includes the embedding layers, and if the end layer equals the total number of layers, it includes the final normalization and head layers.

The next element of the custom class is the forward layer, which carries out one forward pass. If the model's starting layer is 0, it performs embedding at the start. Then, it processes the input through the sharded transformer layers. If the model's end layer equals the total layers, it performs normalization and head operations at the end.

Sharding the LLM into 3 equal parts produces 3 partial models as shown:

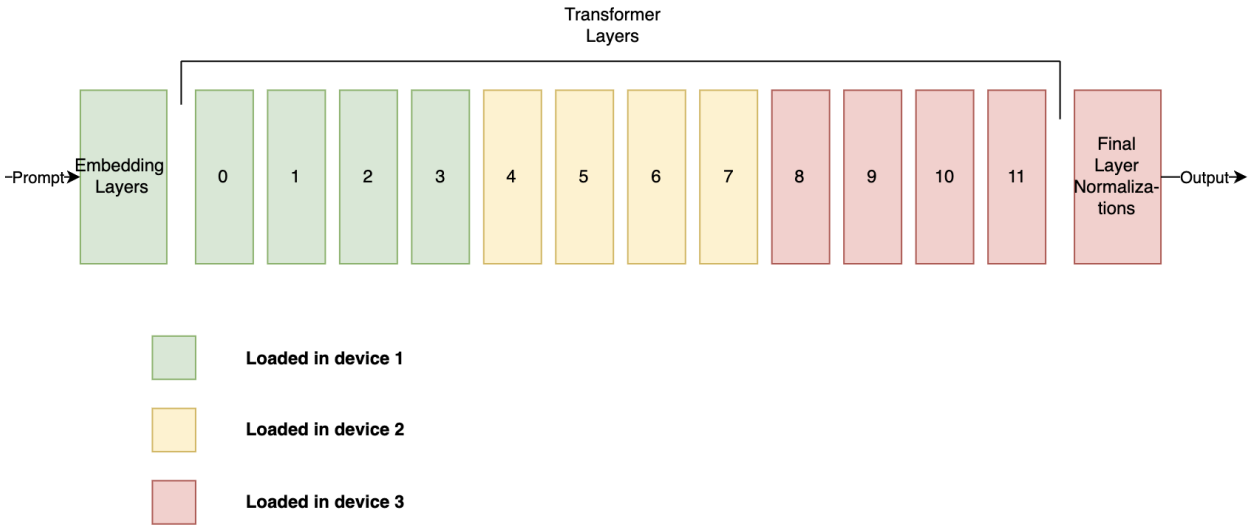


Figure 5.2: Vizualization of different layers in a Model

5.2.2 Splitting the Pretrained Weights

Since the models are split, the pretrained models used for inference should also be split to contain only the weights of the partial model. For this, we created a function `Split_state_dict()`. This function takes the start layer, end layer, and total layers as inputs and removes the transformer weights outside the range (start layer, end layer). The weights of the embedding layers are included if the start layer equals 0, and the head and normalization layer weights are included if the end layer equals the total number of layers.

The keys in the returned weights correspond to the original model's layer names, but when creating the partial model, their keys are adjusted to start from 0. A mapping function called `remap_state_dict()` was used to change state dict keys to match those of the model state dict keys by mapping the keys to 0 to the number of layers in the partial model.

The `remap_state_dict()` function adjusts the layer indices in a given state dictionary for a transformer model. It creates a new dictionary where, for each key-value pair, it checks if the key starts with "transformer.h". If it does, the function extracts the current layer

number, subtracts a specified starting layer from it, and constructs a new key using the updated layer number along with the remaining parts of the original key. If the key does not match the transformer layer pattern, it is added unchanged to the new dictionary. Finally, the function returns this newly constructed state dictionary.

The `filtered_state_dict()` function extracts specific layers from a transformer model's state dictionary based on given start and end layer indices. It creates a new dictionary that includes keys starting with "transformer.h" if their layer numbers fall within the specified range. It also includes keys for the word and positional embeddings if the starting layer is zero, and for the final normalization and language model head if the end layer matches the total layers. Finally, it remaps the layer indices in the filtered dictionary and returns the updated version.

5.3 DHT and Logits Sharing

A distributed hash table (DHT) was implemented using the Hivemind library, a Python library for decentralized deep learning across the Internet. Its intended usage is training one large model on hundreds of computers from different universities, companies, and volunteers.

A server with a public IP was set up using cloud credits as the initial peer to start the Hivemind DHT.

The `dht.get_visible_maddrs()` method retrieves the ID of the peer, which was then used by other peers to connect to the server. Data could then be passed between them using the store and get methods. This setup can be used to pass messages to peers for sequencing and synchronization but could not be used to share large files like tensor logits.

To share the logits, they were either sent directly using websocket established between monitor clients and terminal or browser clients, with the relevant peer being notified using the DHT. The Hivemind DHT also provides fault tolerance, allowing browser or terminal instances to join and leave as they please.

We know from Equation 5.5,

$$\mathcal{M}_\theta = \mathcal{M}_{\theta_n} \circ \mathcal{M}_{\theta_{n-1}} \circ \cdots \circ \mathcal{M}_{\theta_1}$$

After logits x_j is propagated to j^{th} node, it is forward passed from the model it's holding as $\hat{y_j} = \mathcal{M}_{\theta_j}(x_j)$. Then, the output $\hat{y_j}$ is passed to next node N_{j+1} for next computation through websocket. Overall, there exists n total communications between the nodes, $n-1$ from $1 \to 2 \ldots n$ and one from $n \to 1$.

5.4 Llama Implementation

Our use case required fine tuned control over the Llama Architecture. We will need to shard the model into different layers and custom compile the model to run it in the browser

settings. Thus we implemented the Llama architecture from scratch.

Here is an overview of the Llama Architecture:

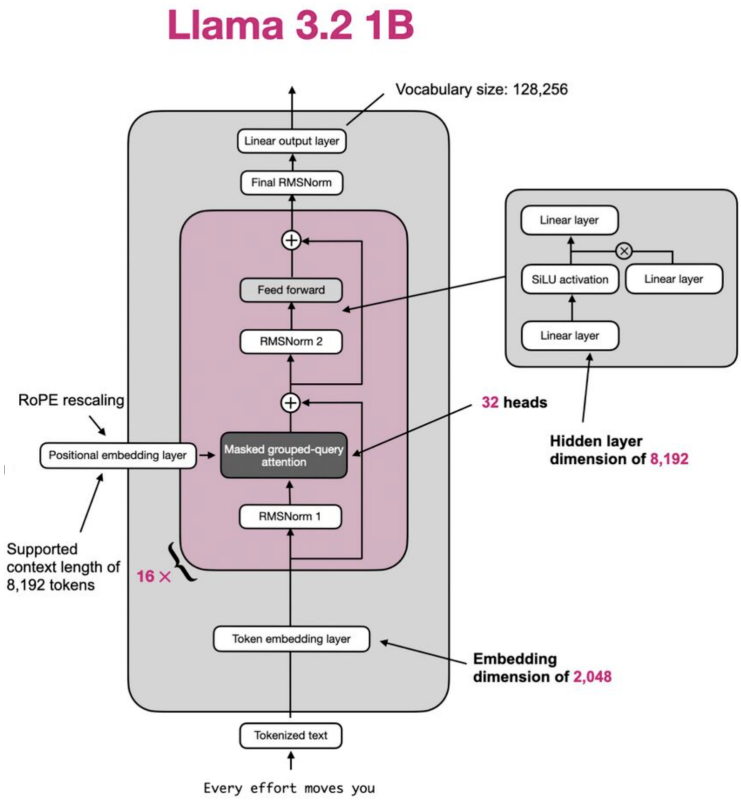


Figure 5.3: Llama3.2-1B Architecture
[26]

This implementation closely follows the original LLaMA model design while allowing for greater flexibility and customization.

The custom architecture includes the following key components:

1. **Feedforward Network**

   Feedforward Network in Llama consists of one up projection layer, one gate layer that applies SiLU function and one down projection at the end.

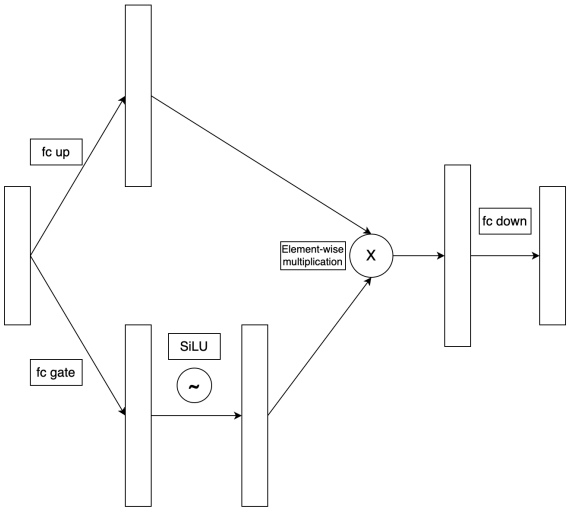


Figure 5.4: SwiGLU FFN
[27]

2. **MultiHead Attention** Llama uses MultiHead Attention to allow for parallel computation of context vectors.

   (a) **GroupedQuery Attention** Grouped-Query Attention (GQA) is an optimization of MultiHead Attention that reduces computational complexity while maintaining model quality.[28] It groups queries into subsets and uses shared keys and values for each group.

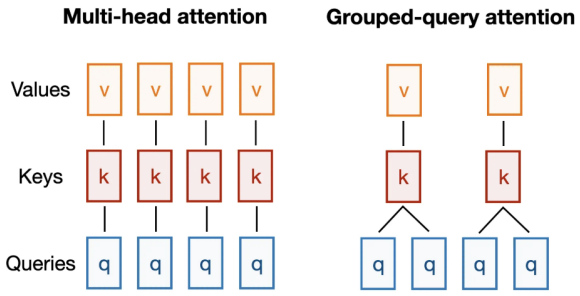


Figure 5.5: Multi-Head Grouped Query Attention
[28]

   (b) **Rotary Positional Embedding** Llama uses Rotary Positional Embedding (RoPE)[29] which is a technique that encodes positional information by rotating the query and key vectors in the attention mechanism. It allows the model to capture relative positional information effectively.

5.5 Sharding Llama Model

After implementing the basic Llama Model, the Llama model was modified so the user can create a partial sharded model. To shard the model, the user would provide the start layer and end layer during the initialization of the model.

The new model will then have only the layers specified during initialization.

This is similar to what we did with the GPT2 model the only difference being that the base model is a custom Llama model.

5.5.1 Weights loading

After creating the model weights, the problem remained of how to load only the weights required since the models weights available in the internet are for the full model. So, we sharded the model weights manually into one layer each and uploaded the layerwise weights into the hugging face. Then we defined a custom .pretrainedfunction that will take the model prefix and download and load the required state dict layerwise.

5.6 Fine Tuning

After creating model and loading pretrained weights, we also implemented fine tuning. For fine tuning we looked at Parameter Efficent Fine Tuning(PEFT) techniques and more specifically Low Rank Adaptation(LoRA) fine tuning.

5.6.1 LoRA

A LoRALayer class was defined to create low-rank adaptations of linear layers. It defines two matrices A and B whose resulting dimension after multiplication matches that of the parameter of the linear layer that lora will work with. The dimension of A is set to (indim,rank) and that of B to (outdim,rank). Rank determines how heavy we want the fine tuning to be. The weights of A is initialized to random standard distribution and B to 0 so that AB=0 initially.

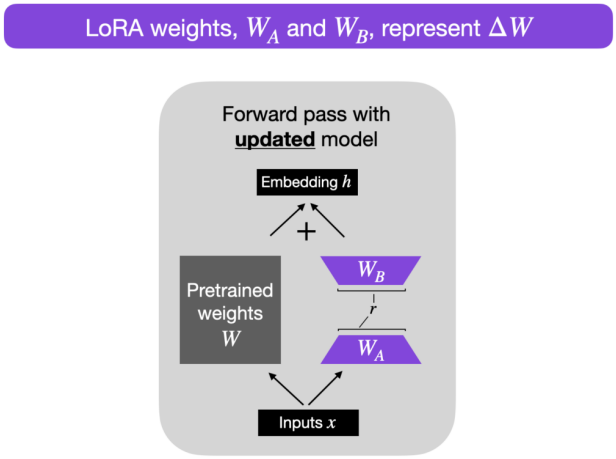


Figure 5.6: LoRA
[30]

LinearWithLoRA wraps existing linear layers with LoRA functionality.

5.6.2 Setting up model for LoRA fine tuning

- Instantiate the custom llama model
- Load the pretrained weights
- Freeze all the parameters in the model
- Replace the desired linear layers with LinearWithLoRA

5.6.3 Preparing DataSets and Dataloaders

For the experimentation purposes, we used the web scraped entrance exam question, answers. This data was cleaned to form a dataset in json format containing keys: instruction, question, answer, and explanation.

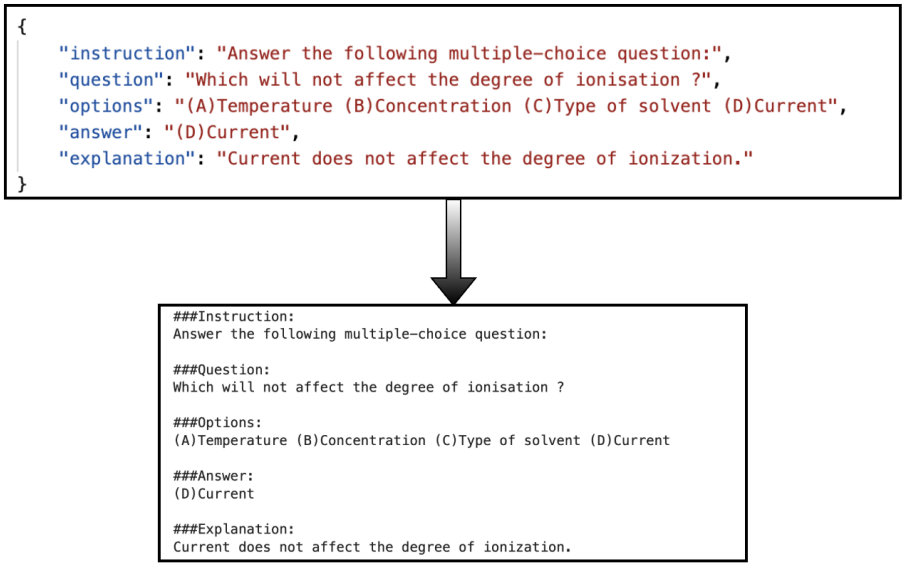


Figure 5.7: Alpaca Style Prompt Formatting

We used alpaca style prompt formatting to transfer the dataset into appropriate form for finetuning.

An InstructionDataset class was created to handle the custom dataset format. A custom collate function pads and prepares batches for training. DataLoaders are created for train, validation, and test sets.

5.6.4 Distributed FineTuning

Finetuning has 4 main steps:

1. Forward Pass where graph of the computations is calculated
2. Loss calculation at the final layer using the output and target
3. Calculating the gradients
4. Changing the parameter's weights

The first two steps can be achieved easily using the inference implementations already achieved which involved passing the intermediate nodes to the next node.

But this is not sufficient for step 3 since training requires passing the logits to the next node during forward pass and gradients to the previous node during the backward pass.

After the last layer calculates the loss from the target and the output, it will then calculate the gradients of the weights all the way back to the input logits it received. This gradient

will be then sent back to the previous node as the gradient for the output of the previous node.

We choose a recursive function approach for each training step. The initial node will call the step function produce its output and call the step function of the next node with its output. The last node will then return the gradients of the input it received as for the step function called by its previous node on it. The previous node will use this gradients to calculate the gradients of its relevant parameters and change their weights. This will go back to the initial node sequentially updating each node's weight in the process.

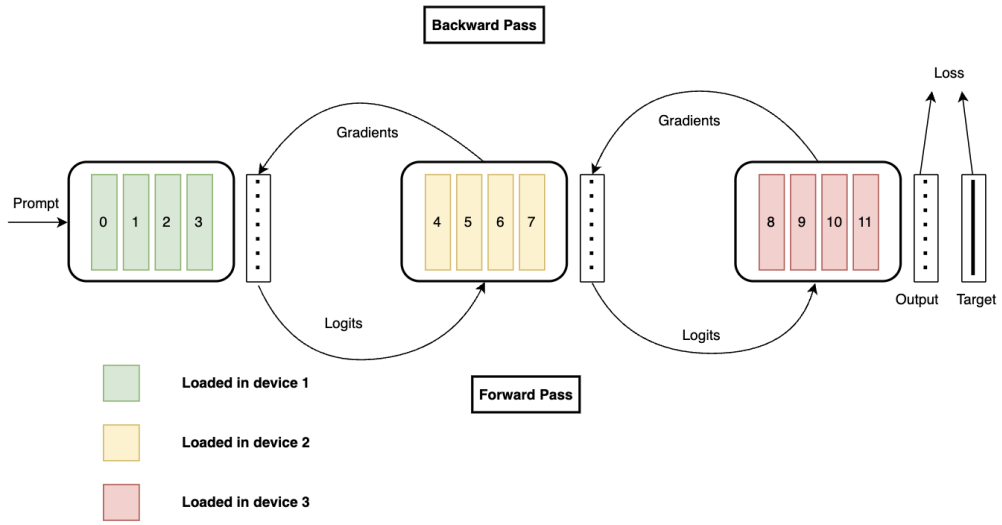


Figure 5.8: A single step in fine-tuning

For fine tuning of LLMs, cross entropy loss function was choosen.

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\log \hat{y}_{i,t}^{(y_{i,t})} \tag{5.6}$$

where $\hat{y}_{i,t}$ represents the probability with which the model predicts the next token is the target token for i^{th} batch and t^{th} place in the sequence. N is the total batch size.

5.7 Multi-Layer WebSocket Communication System

We developed a robust multi-layer WebSocket communication system that supports dynamic message processing and real-time performance monitoring. Using asynchronous programming, our solution seamlessly integrates both server-side and client-side components. This architecture is designed to support multiple clients operating in distinct processing layers while efficiently managing and tracking the overall communication flow.

5.7.1 Server-Side Implementation

Worker and Connection Management

Worker Class: We defined a `Worker` class to encapsulate the properties of each client, including a unique identifier and a busy status flag. This abstraction helps us monitor whether a worker is currently engaged in processing a task.

Connection Manager: A dedicated `ConnectionManager` maintains a dictionary of active WebSocket connections. It provides methods to add new connections, remove disconnected clients, and robustly handle binary message exchanges with comprehensive error handling.

Worker Manager: To balance load distribution, our `WorkerManager` organizes workers into predefined processing layers. It supports adding workers to specific layers, removing inactive workers, and selecting the least populated layer when dispatching tasks.

Message Processing Logic

At the heart of our system lies a central message processing loop that iterates over a fixed number of cycles. In each iteration, the system:

1. Selects an available worker from each layer.
2. Constructs a payload based on the processing layer (e.g., initial prompt, accumulated data such as logits).
3. Sends the payload to the worker, waits for the response, and updates the message content for the next iteration.

This iterative process refines the input prompt across multiple layers until the final output is achieved.

WebSocket Connection Handling

Our WebSocket logic includes:

1. Accepting incoming connections and executing a binary handshake that encodes both the processing layer number and the client's unique ID.
2. Parsing handshake data to register the client with the `ConnectionManager` and `WorkerManager`.
3. Maintaining the connection with a keep-alive loop, ensuring resource cleanup and proper deregistration upon client disconnection.

5.7.2 Client-Side Implementation

Establishing the WebSocket Connection

Using `websockets` library, our client-side code establishes an asynchronous connection with the server. Upon connection, the client sends a binary-encoded handshake containing its designated processing layer number and unique client ID.

Processing Loop and Message Handling

After the handshake, the client enters a processing loop where it:

1. Receives messages from the server and logs the duration of data receipt.
2. Differentiates between binary and text messages, handling each appropriately.
3. Performs decoding and parsing, converting bytes to tensors and vice versa, while generating intermediate logits or model outputs.
4. Records detailed timestamps for each step (receiving, processing, decoding, and sending) to facilitate performance monitoring and debugging.

Performance Monitoring and Logging

Throughout the entire communication cycle, precise timestamps are logged for every message transfer. This detailed monitoring framework enables us to identify potential performance bottlenecks and optimize the communication flow across the system.

Communication Protocol and Error Handling

A standardized binary handshake initiates every connection, encoding both the processing layer and client ID to ensure mutual understanding between the server and client. This robust protocol, coupled with rigorous error handling, ensures smooth communication and swift recovery from potential errors.

5.8 Experimental Setup for PEFT

Parameter	1B Model	3B Model
Model and LoRA Settings		
Model Architecture	LLaMA-3.2-1B	LLaMA-3.2-3B
Prefix	pAce576/Llama-3.2-1B-sharded	pAce576/Llama-3.2-3B-sharded
No of Shards	2 (0-7, 8-15)	3 (0-7, 8-15, 16-27)
LoRA Rank (r)	8	8
LoRA Alpha	1	1
LoRA Dropout	0.05	0.05
Target Modules	`q_proj, v_proj, mlp_proj`	`q_proj, v_proj, mlp_proj`
Trainable Parameters	∼4.6M	∼9.4M
Training Configuration		
Batch Size	3	3
Learning Rate	5e-5	5e-5
Optimizer	AdamW	AdamW
Precision	float16	float16
Dataset and Environment		
Dataset Name	Custom IoE Entrance Question Set	Custom IoE Entrance Question Set
Training Hardware	1 × RTX 3090, 1 x A6000	1 × RTX 3090, 1 x A6000
LoRA Backend	bitsandbytes	bitsandbytes

Table 5.1: Experimental Setup for LoRA Fine-Tuning of 1B and 3B Models

6. Results

We compared the inference and training times using full model in a single device, with RAM offloading and with model parallelism(our framework).

6.1 Experimentation

6.1.1 Experiment with Different Bandwidths

The llama 3.3 1B was used as the base model for inference and training purpose. The T4 16GB GPU was used for the inference and training purposes of llama 3.3 1B model: The llama 3.3 1B was splitted into 3 shards:

1. Model1: 0-5 layers
2. Model2: 5-10 layers
3. Model3: 10-16 layers

For the full model purpose, the whole llama model was loaded and inference and training were run on it.

For inference and training with RAM off loading, the model1 was loaded into gpu first, logits were calculated, and then it was offloaded to cpu, then model2 was loaded, logits calculated and offloaded and so on for each computation required from the sharded model.

For model parallelism, since we were having network instability issues, we used a simulated environment to calculate the inference and training times. For this, we loaded all three shards in the same GPU, then before passing logits or gradients to the next or previous shard, we calculated the size of the data we would need to send and added a delay function that will stop the execution for time required to send that size of data over some constant(1MBPS, 5MBPS, 10MBPS) data speed thus simulating real data transmission.

Graphs

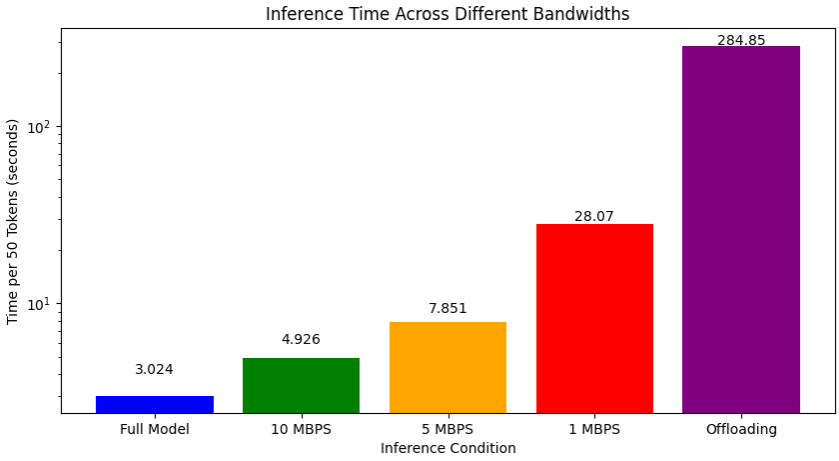


Figure 6.1: Average inference time per 50 token

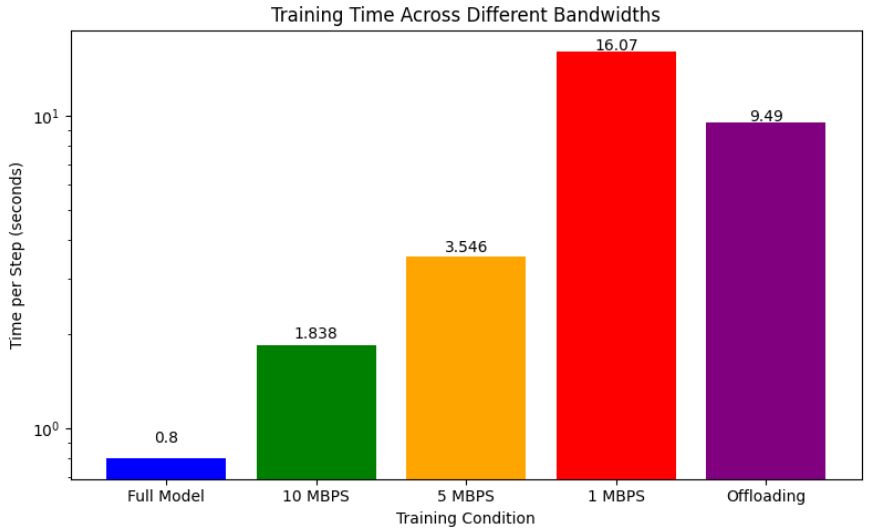


Figure 6.2: Average training time per step

6.1.2 Fine-Tuning

We finetuned 2 different llama models namely llama 3.3 1B and llama 3.3 3B using a custom-crafted dataset scraped from the internet, with more details in the section 5.8. The models were divided into 2 and 3 shards respectively in the layers as:

- llama 1B:

  1. Model1: 0-7 layers

2. Model2: 8-15 layers

- llama 3B:

1. Model1: 0-7 layers
2. Model2: 8-15 layers
3. Model3: 16-27 layers

Graphs

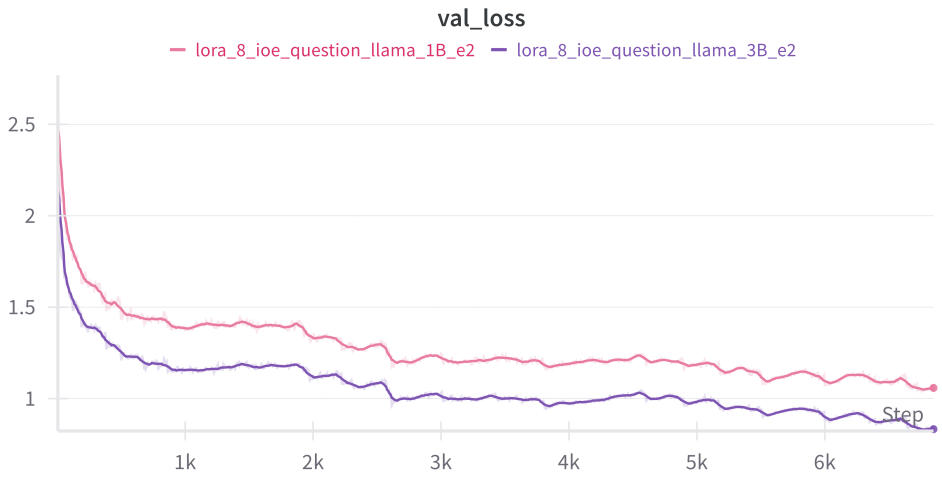


Figure 6.3: Validation Loss per step

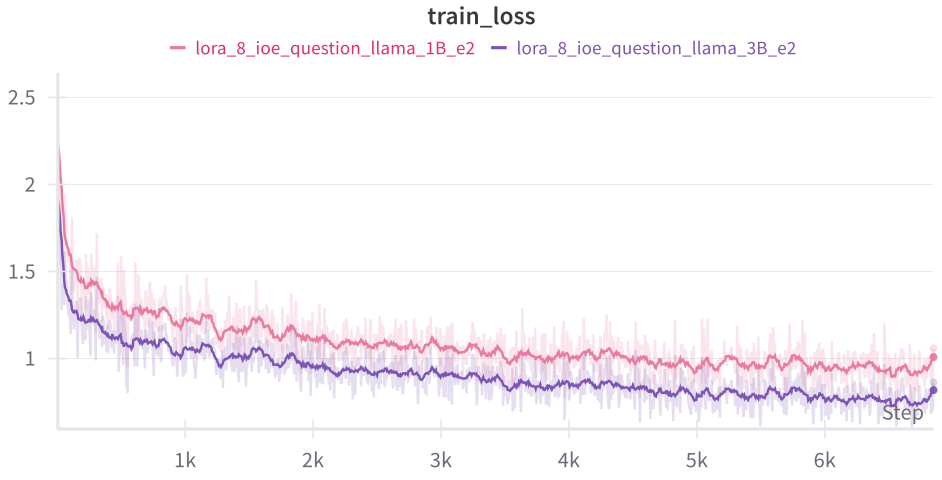


Figure 6.4: Training Loss per step

The loss converged as intended. This is since the gradient updates were successful and the model was fine-tuned on our experimental setup.

6.1.3 Discussion

The chart obtained for inference time per 50 tokens suggested that model parallel approach is faster than model offloading using single GPU instance even with 1 MBPS. For our experiment instance, it was 10 times better than model offloading. And 1 MBPS is a speed that is easily achievable in most network/ internet environments. The average global internet speed is 110Mbps which is equivalent to 13MBPS according to Ookla Speedtest Global Index, Jan 2024.

For fine-tuning, our approach outperformed the offloading approach if the network speed is about 5MBPS but was performed worse than offloading for the 1MBPS case. The performance of the model parallel approach is dependent on the data size of the logits and gradient data whereas the offloading depends on the ratio between the size of the model and the RAM space along with the speed of the PCIe connector(which is the speed bottleneck in the offloading approaches).

Model Parallelism and Network Speed: Inference Time

Assumptions

- Data type: `fp16`
- Only network bandwidth delay is considered (latency, jitter, packet losses are neglected).

Let:

$$N = \text{Number of tokens}$$
$$K = \text{Hidden dimension (logits dimension)}$$
$$V = \text{Network speed in Bytes per second (Bps)}$$
$$I = \text{Number of nodes}$$

Each token has a data size of:

$$\text{Data per token} = K \times 2 \text{ bytes} \quad \text{(fp16)}$$

Total data to be transferred per inference cycle:

$$\text{Total Data} = N \times K \times I \times 2 \text{ bytes}$$

Thus, time spent on communication:

$$T_C = \frac{N \times K \times I \times 2}{V}$$

Example values from experiment:

$$N = 25$$
$$K = 2048$$
$$I = 3$$
$$V = 1024 \times 1024 \text{ Bps} = 1 \text{ MBPS}$$

$$T_C = \frac{25 \times 2048 \times 3 \times 2}{1024 \times 1024} \approx 0.29 \text{ seconds}$$

Offloading and Model Size: Inference Time

*Assumptions

$$M = \text{Model size in bytes}$$
$$R = \text{Available VRAM}$$
$$U = \text{PCIe offloading speed (Bps)}$$
$$S = \left\lceil \frac{M}{R} \right\rceil = \text{Number of shards}$$
$$Q = \frac{M}{S} = \text{Size per shard}$$

Time spent in offloading:

$$T_O = \frac{2 \times S \times Q}{U} = \frac{2 \times S \times M}{S \times U} = \frac{2M}{U}$$

Example values from experiment:

$$M = 4 \text{ GB} = 4 \times 1024^3 \text{ bytes}$$
$$U = 12 \text{ GB s}^{-1}$$

$$T_O = \frac{2 \times 4 \times 1024^3}{12 \times 1024^3} = \frac{8}{12} \approx 0.667 \text{ seconds}$$

Comparison Ratio

Define ratio R of communication time to offloading time:

$$R = \frac{T_C}{T_O} = \left(\frac{N \times K \times I}{M}\right)\left(\frac{U}{V}\right)$$

If we take:

$$\frac{U}{V} = \frac{11 \times 1024^3}{12 \times 10^6} \approx 1000$$

Then:

$$R < 1 \Rightarrow N \times K \times I \times U < M \times V \Rightarrow NKI < M \left(\frac{V}{U}\right)$$

This shows our approach is better when the total number of fp16 tensor outputs per node is significantly less than the model size, adjusted by the bandwidth ratio(about 1000 times less). It is noteworthy that this is an approximate formed using assumptions that may not always hold true. But it does provide a rule of thumb figure to work with to choose between model offloading and model parallelism.

7. Conclusion

De-Llmit offers an innovative approach to large language model (LLM) inference by distributing computational tasks across multiple devices. This makes advanced AI more accessible to a wider audience. By leveraging decentralized technologies, De-Llmit not only overcomes hardware limitations but also invites more contributors to engage with LLM technologies. This project fosters collaboration and effectively utilizes the resources of a distributed network.

The project enhances accessibility and encourages collaboration within the decentralized network, where participants contribute their computational resources for model inference and fine-tuning. The distributed nature of the system ensures scalability, fault tolerance, and lower operational costs while maintaining high performance. Moreover, De-LlMIT integrates advanced technologies such as model sharding and Distributed Hash Tables (DHT) to efficiently coordinate the flow of computation across various nodes, ensuring optimal utilization of network resources.

Overall, De-LlMIT fosters a more inclusive and collaborative AI ecosystem, inviting broader engagement in LLM technologies. By addressing critical challenges like resource distribution, network reliability, and accessibility, De-LlMIT paves the way for scalable, decentralized AI solutions that can reach a much wider audience without compromising on performance or security.

REFERENCES

[1] J. Sevilla, J. Heim, M. Ho, and et al., "Compute trends across three eras of machine learning," *arXiv preprint arXiv:2202.05924*, 2022. [Online]. Available: https://arxiv.org/abs/2202.05924.

[2] A. Bambhaniya et al., *Demystifying platform requirements for diverse llm inference use cases*, 2024. arXiv: 2406.01698 [cs.AR]. [Online]. Available: https://arxiv.org/abs/2406.01698.

[3] A. Borzunov, M. Ryabinin, and I. Titov, "Petals: Collaborative inference and fine-tuning of large models," *arXiv preprint arXiv:2209.01188*, 2022. [Online]. Available: https://arxiv.org/html/2405.10313v2.

[4] J. Verbraeken, M. Wolting, J. Katzy, J. Kloppenburg, T. Verbelen, and J. S. Rellermeyer, "A survey on distributed machine learning," *CoRR*, vol. abs/1912.09789, 2019. arXiv: 1912.09789. [Online]. Available: http://arxiv.org/abs/1912.09789.

[5] M. Ryabinin et al., *Hivemind: Decentralized Deep Learning in PyTorch*, Online, Apr. 2020. [Online]. Available: https://github.com/learning-at-home/hivemind.

[6] M. Ryabinin and A. Gusev, "Towards crowdsourced training of large neural networks using decentralized mixture-of-experts," in *Advances in Neural Information Processing Systems*, vol. 33, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/file/25ddc0f8c9d3e22e03d3076f98d83cb2-Paper.pdf.

[7] A. L. Beberg, D. L. Ensign, G. Jayachandran, S. Khaliq, and V. S. Pande, "Folding@home: Volunteer distributed computing," in *IEEE International Symposium on Parallel & Distributed Processing (IPDPS)*, Stanford University. Available at https://cs.stanford.edu/people/beberg/Folding@home2009.pdf, 2009.

[8] S. Laskaridis, S. I. Venieris, N. D. Lane, and D. Lucanin, "The future of consumer edge-ai computing," *arXiv preprint arXiv:2210.10514*, 2022. [Online]. Available: https://arxiv.org/pdf/2210.10514.

[9] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020. DOI: 10.1109/MSP.2020.2975749.

[10] T. Ben-Nun and T. Hoefler, "Model-distributed machine learning," *ACM Computing Surveys (CSUR)*, vol. 52, no. 4, pp. 1–36, 2019.

[11] P. Maymounkov and D. Mazieres, "Kademlia: A peer-to-peer information system based on the xor metric," *Proceedings of the 2002 ACM SIGCOMM Workshop on Practice and Theory of Incentives in Networked Systems*, pp. 53–65, 2002.

[12] X. Wang, L. Liu, B. Yang, C. Li, and Y. Zhang, "P2p networking and dhts in decentralized systems," *IEEE Transactions on Network and Service Management*, vol. 17, no. 3, pp. 1234–1250, 2020.

[13] T. B. Brown et al., "Scaling up language models: Gpt-3," *Proceedings of the 2020 Conference on Neural Information Processing Systems (NeurIPS 2020)*, pp. 1–53, 2020.

[14] J. Kim et al., "Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization," *arXiv preprint arXiv:2305.14152*, 2023, Published at NeurIPS 2023. Camera-ready version. DOI: 10.48550/arXiv.2305.14152. arXiv: 2305.14152 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2305.14152.

[15] O. Weng, "Neural network quantization for efficient inference: A survey," *arXiv preprint arXiv:2112.06126*, 2021.

[16] X. Li, X. Xu, and X. Liu, "Memory-efficient training of transformers," *Journal of Machine Learning Research*, vol. 21, no. 1, pp. 1–27, 2020.

[17] Y. Jiang, M. Li, Q. Xu, X. Zhao, J. Wang, and G. Lu, "Webinf: Accelerating webgpu-based in-browser dnn inference via adaptive model partitioning," *IEEE Transactions on Parallel and Distributed Systems*, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10476167.

[18] M. Sato, R. Sasaki, D. Saito, and D. Morikawa, "Webdnn: Fastest dnn execution framework on web browser," in *Proceedings of the 26th International Conference on World Wide Web Companion*, ACM, 2017, pp. 1157–1166. [Online]. Available: https://doi.org/10.1145/3123266.3129394.

[19] D. Smilkov et al., "Tensorflow.js: Machine learning for the web and beyond," *CoRR*, vol. abs/1901.05350, 2019. arXiv: 1901.05350. [Online]. Available: http://arxiv.org/abs/1901.05350.

[20] L. Wu, X. Li, R. Zhao, W. Lu, J. Xu, and F. Xue, "A blockchain-based model with an incentive mechanism for cross-border logistics supervision and data sharing in modular construction," *Journal of Cleaner Production*, vol. 375, p. 133 460, 2022, ISSN: 0959-6526. DOI: https://doi.org/10.1016/j.jclepro.2022.133460. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0959652622030414.

[21] J. Wen, Z. Zhang, Y. Lan, Z. Cui, J. Cai, and W. Zhang, "A survey on federated learning: Challenges and applications," *International Journal of Machine Learning and Cybernetics*, vol. 14, no. 2, pp. 513–535, 2023, ISSN: 1868-808X. DOI: 10.1007/s13042-022-01647-y. [Online]. Available: https://doi.org/10.1007/s13042-022-01647-y.

[22] L. Yang, Y. Lu, J. Cao, J. Huang, and M. Zhang, "E-tree learning: A novel decentralized model learning framework for edge AI," *CoRR*, vol. abs/2008.01553, 2020. arXiv: 2008.01553. [Online]. Available: https://arxiv.org/abs/2008.01553.

[23] A. Borzunov et al., "Distributed inference and fine-tuning of large language models over the internet," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36, Curran Associates, Inc., 2023, pp. 12 312–12 331. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/28bf1419b9a1f908c15f6195f58cb865-Paper-Conference.pdf.

[24] A. Melnikov and I. Fette, *The WebSocket Protocol*, RFC 6455, Dec. 2011. DOI: 10.17487/RFC6455. [Online]. Available: https://www.rfc-editor.org/info/rfc6455.

[25] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762.

[26] L. V. D. Maaten and L. Team, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024. [Online]. Available: https://arxiv.org/abs/2407.21783.

[27] N. Shazeer, "Glu variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020. [Online]. Available: https://arxiv.org/abs/2002.05202.

[28] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore: Association for Computational Linguistics, 2023, pp. 4895–4901. DOI: 10.18653/v1/2023.emnlp-main.298.

[29] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[30] E. J. Hu et al., "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: https://arxiv.org/abs/2106.09685.